



BBDD – Diseño Físico

⚙ Estado	Completado
👤 Propietario	🇪🇸 Oscar Cerlanga [Ⓜ] María Paradela [Ⓡ] Raúl Leciñena
🕒 Prioridad	Alta
☰ Categoría	
☰ Equipo	

1. Descripción del diseño físico

El diseño físico representa la implementación concreta del modelo lógico de la base de datos utilizando PostgreSQL como sistema gestor de bases de datos.

En este nivel se definen las tablas, los tipos de datos, las claves primarias, claves foráneas y restricciones necesarias para materializar las entidades y relaciones del sistema de gestión de reservas de equipos de sonido y DJs.

El objetivo del diseño físico es transformar el modelo lógico en una estructura que pueda ser ejecutada directamente por el SGBD.

Las entidades implementadas en la base de datos son:

- Usuario
- Reserva
- Equipo
- DJ
- Presupuesto
- Detalle_Presupuesto
- Factura
- Pago
- Contacto

- Datos_Empresa

Además, se implementan dos tablas intermedias para resolver relaciones muchos a muchos:

- Incluye (Reserva–Equipo)
- Contrata (Reserva–DJ)

También se definen tipos ENUM para representar los posibles estados de una reserva, del presupuesto, el rol de usuario y el tipo de contacto. El método de pago se almacena como texto libre (`VARCHAR`) para reflejar el valor devuelto por la API de pagos externa.

Además, se implementa la tabla `datos_empresa` como tabla de configuración con una única fila, para almacenar los datos fiscales del emisor necesarios en la generación de presupuestos y facturas, incluyendo el IBAN para pagos por transferencia.

2. Consideraciones del diseño físico

- Se utiliza PostgreSQL como sistema gestor de bases de datos, desplegado a través de Supabase.
- Las claves primarias de las tablas internas se implementan mediante `INTEGER GENERATED ALWAYS AS IDENTITY`, sintaxis estándar de PostgreSQL 15+ compatible con Supabase (SERIAL está deprecado).
- La tabla `usuario` usa `UUID` como clave primaria, vinculada directamente a `auth.users(id)` de Supabase Auth. Esto permite integrar la autenticación nativa de Supabase sin duplicar la gestión de contraseñas. **La contraseña la gestiona Supabase Auth internamente; no se almacena en la tabla `usuario`.**
- El registro y login de usuarios se realiza desde Express mediante el SDK de Supabase (`supabase.auth.signUp` / `supabase.auth.signInWithPassword`). Al registrarse, Supabase devuelve un UUID que se usa como `id_usuario` al insertar en la tabla `usuario`.
- Las relaciones entre entidades se implementan mediante claves foráneas con nombre explícito (`CONSTRAINT fk_...`).
- Todas las claves foráneas incluyen una política `ON DELETE` explícita: `RESTRICT` para proteger datos dependientes, `CASCADE` para eliminar registros

subordinados cuando se borra el registro padre (tablas intermedias y detalles), y `SET NULL` en `reserva → usuario` para conservar el historial de reservas cuando un usuario elimina su cuenta.

- Las tablas `incluye` y `contrata` utilizan claves primarias compuestas para representar relaciones muchos a muchos.
- La tabla `contacto` se implementa como una entidad independiente para registrar formularios enviados desde la navegación pública sin requerir autenticación.
- El campo `id_usuario` en `reserva` es nullable para soportar dos escenarios: reservas creadas por el usuario desde la web (con `id_usuario`) y reservas manuales creadas por el admin, por ejemplo vía telefónica (con `id_usuario = NULL`).
- La tabla `reserva` incluye los campos `cliente_dni_nie_cif`, `cliente_nombre`, `cliente_email`, `cliente_telefono`, `cliente_direccion`, `cliente_codigo_postal`, `cliente_localidad` y `cliente_provincia` como `NOT NULL`. El campo `cliente_dni_nie_cif` admite DNI, NIE o CIF para dar cobertura tanto a clientes particulares como a empresas. La dirección completa del cliente (dirección, código postal, localidad y provincia) se almacena directamente en la reserva para garantizar que toda reserva tenga siempre identificado al cliente de forma completa, independientemente de si existe un usuario registrado. Cuando un usuario registrado crea la reserva, el backend copia automáticamente todos sus datos desde la tabla `usuario`; cuando el admin crea una reserva manual, introduce los datos directamente desde el panel de administración. Esto permite generar presupuestos y facturas con los datos completos del cliente en cualquier punto del flujo y facilita las automatizaciones con N8N.
- La tabla `factura` no incluye datos del cliente de forma directa. Al generar una factura, los datos del cliente se obtienen navegando las relaciones `factura → presupuesto → reserva`, donde los campos `cliente_*` siempre están disponibles (son `NOT NULL` y no se ven afectados por la eliminación de la cuenta del usuario). Esto evita redundancia y simplifica el modelo.
- Un usuario solo puede borrar su cuenta si no tiene reservas en estado `pendiente` o `confirmada`. Esta comprobación se realiza desde Express antes de llamar a `supabase.auth.admin.deleteUser()`. El `CASCADE` de `auth.users → usuario` elimina la fila de `usuario`, y el `SET NULL` de `reserva → usuario` deja las reservas

históricas con `id_usuario = NULL`, conservando los datos del cliente intactos en los campos `cliente_*` de la reserva.

- La tabla `datos_empresa` almacena los datos fiscales de la empresa emisora (nombre, CIF, dirección, código postal, localidad, provincia, email, teléfono e IBAN). El campo `iban` se utiliza para mostrar los datos bancarios en presupuestos y facturas cuando el cliente opta por pago mediante transferencia. Se implementa como tabla de configuración con una única fila, restringida mediante `CHECK (id_empresa = 1)`. No tiene relación con otras tablas mediante claves foráneas; los datos se consultan directamente al generar presupuestos y facturas, tanto desde el backend como desde las automatizaciones con N8N.
- La fecha de contacto incluye un valor por defecto `CURRENT_DATE` para registrar automáticamente la fecha de envío del formulario.
- La tabla `keepalive` se mantiene para evitar la inactividad de la base de datos en Supabase. Contiene una única fila y el campo `last_ping` es de tipo `TIMESTAMPZ` con valor por defecto `NOW()`. La actualización de este campo se realiza mediante una automatización en N8N que ejecuta `UPDATE keepalive SET last_ping = NOW() WHERE id = 1` diariamente, dejando que Postgres sea siempre la fuente del tiempo.
- Los estados y tipos categóricos se implementan mediante tipos `ENUM` para restringir los valores posibles a los definidos en el modelo. El campo `metodo_pago` de la tabla `pago` es una excepción: se usa `VARCHAR(50)` en lugar de `ENUM` porque su valor lo dicta la API de pagos externa (por ejemplo, Stripe), lo que lo hace incompatible con un conjunto fijo de valores definidos en base de datos.
- La tabla `usuario` incluye los campos `direccion VARCHAR(200)`, `codigo_postal VARCHAR(10)`, `localidad VARCHAR(100)` y `provincia VARCHAR(100)`, todos `NOT NULL`. Estos datos se copian automáticamente a los campos `cliente_*` de la reserva cuando un usuario registrado crea una reserva, y se utilizan como fuente para la generación de presupuestos y facturas.
- El campo `metodo_pago` de la tabla `pago` se usa `VARCHAR(50)` en lugar de `ENUM` porque su valor puede ser dictado por la API de pagos externa, lo que lo hace más flexible ante posibles integraciones.